

PART THREE

程序的基本编写方法

The Basic Programming Method

- 每个程序都有统一的运算模式，即输入数据、处理数据和输出数据，这种朴素运算模式形成了程序的基本编写方法，即IPO方法
 - **I**ntput：输入数据
 - **P**rocess：处理数据
 - **O**utput：输出数据

■ Input : 输入数据

- 输入 (Input) 是一个程序的开始
- 程序要处理的数据有多种来源，形成了多种输入方式，包括：文件输入、网络输入、控制台输入、交互界面输入、随机数据输入、内部参数输入等
- 典型输入设备：键盘、鼠标、麦克风、摄像头



■ Process：处理数据

- 处理（Process）是程序对输入数据进行计算产生输出结果的过程
- 计算问题的处理方法统称为“算法”，它是程序最重要的组成部分。可以说，算法是一个程序的灵魂

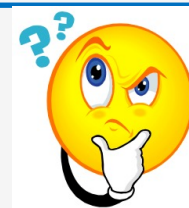


■ Output : 输出数据

- 输出 (Output) 是程序展示运算成果的方式
- 程序的输出方式包括：控制台输出、图形输出、文件输出、网络输出、操作系统内部变量输出等
- 典型输出设备：显示器、打印机、扬声器



■ 是否存在没有输入输出的程序呢？



- 存在，例如，无限循环

```
*untitled*
File Edit Format Run Options Window Help
while(True):
    a=1
```

- 尽管无限循环没有输入输出，但通过不间断执行，可用来辅助测试CPU或系统性能；但这类没有输入输出的程序在功能上十分有限，仅在特殊情况下使用

- IPO不仅是程序设计的基本方法，也是描述计算问题的方式
- 微实例：圆面积的计算

输入：圆半径radius

处理：计算圆面积 $\text{area} = \pi * \text{radius} * \text{radius}$

输出：圆面积area

- ◆ 问题的IPO描述实际上是对一个计算问题的输入、输出和求解方式的自然语言描述，能够帮助初学程序设计者理解程序设计的开始过程，进而建立设计程序的基本概念
- ◆ IPO方法是非常基本的程序设计方法，随着学习的深入，之后会从设计思想入手介绍程序设计方法论，建立对较大规模程序框架的理解

使用计算机解决问题的步骤

- 编写程序的目的是“使用计算机解决问题”，一般分为6个步骤：
 - 分析问题：分析问题的计算部分
 - ✓ 必须明确，计算机只能解决计算问题，即解决一个问题的计算部分
 - ✓ 对计算问题的不同理解会产生不一样的程序，如求解线性方程组可用高斯消元法，也可用LU分解法
 - 划分边界：划分问题的功能边界
 - ✓ 计算机只能完成确定性的计算功能，需明确问题的输入、输出和处理过程中的要求
 - ✓ 可以利用IPO方法辅助分析问题的计算部分，给出问题的IPO描述

使用计算机解决问题的步骤

- 编写程序的目的是“使用计算机解决问题”，一般分为6个步骤：
 - 设计算法：设计问题的求解算法
 - ✓ 简单的程序功能输入和输出的关系比较直观，程序结构比较简单，直接选择或设计算法即可；对于复杂的程序功能，需要利用程序设计方法将“大功能”划分成“小功能”
 - 编写程序：编写问题的计算程序
 - ✓ 选择一门编程语言，将程序结构和算法设计用编程语言来实现
 - ✓ 任何通用编程语言都可以用来解决计算问题，但不同编程语言的运行性能、可读性、可维护性、开发周期和调试等方面有很大不同
 - ◆ Python语言相比C语言在运行性能上略有逊色，但Python在可读性、可维护性和开发周期等方面比C语言有优势

使用计算机解决问题的步骤

- 编写程序的目的是“使用计算机解决问题”，一般分为6个步骤：
 - 调试测试：调试和测试程序
 - ✓ 通过单元测试和集成测试评估程序运行结果的正确性
 - ✓ 程序正确运行后，可以采用更多测试发现程序在各种情况下的特点，如压力测试、安全性测试等
 - 升级维护：适应问题的升级维护
 - ✓ 任何程序都有它的历史使命，在这个使命结束前，随着功能需求、计算需求和应用需求的不断变化，程序将不断地升级维护，以适应这些变化

使用计算机解决问题的步骤

■ 实例：身份证号码的性别判断

11010020190101123X

男？

女？

➤ 分析问题：

- ✓ 如何输入一个身份证号码
- ✓ 如何从身份证号码判断性别
- ✓ 如何输出结果

控制台输入/输出

➤ 确定功能：

- ✓ 获取身份证号码的倒数第二位，判断奇偶性，奇数为男性，偶数为女性



使用计算机解决问题的步骤

■ 实例：身份证号码的性别判断

➤ ⚠ `input()` 函数返回的是字符串，而非数字

```
>>> x = input('请输入一个数字：')
```

请输入一个数字： 5

```
>>> type(x)
```

```
<class 'str'> # 是字符串，不是数字！
```

```
>>> x + 1
```

```
TypeError: can only concatenate str (not "int") to str
```

使用计算机解决问题的步骤

■ 实例：身份证号码的性别判断

➤ 设计算法：

- ✓ 使用字符串操作符获取倒数第二位字符
- ✓ 将其强制转换为整型数字
- ✓ 通过求余符号判断奇偶性

➤ 编写程序：

```
# 身份证号码的判断程序
id_code = input("请输入18位身份证号码:")

number = int(id_code[-2])

if number % 2 == 0:
    print("女")
else:
    print("男")
```

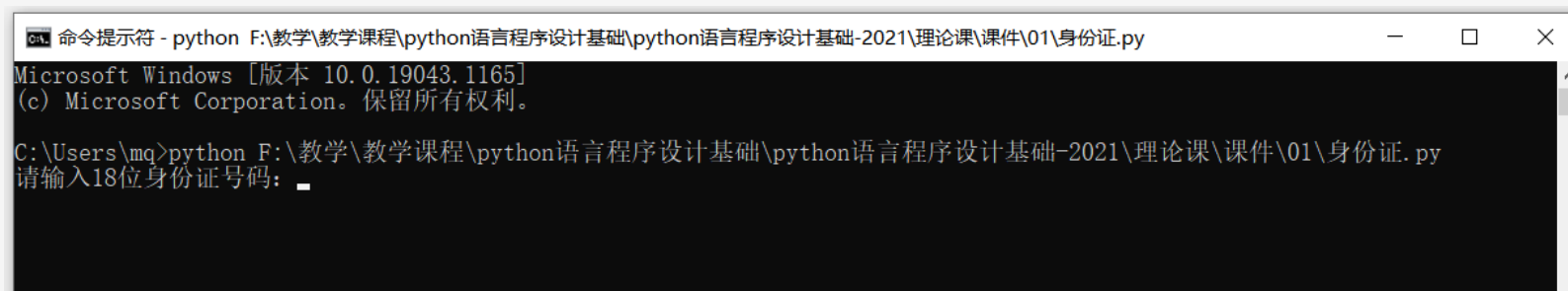
使用计算机解决问题的步骤

■ 实例：身份证号码的性别判断

➤ 调试、运行程序：

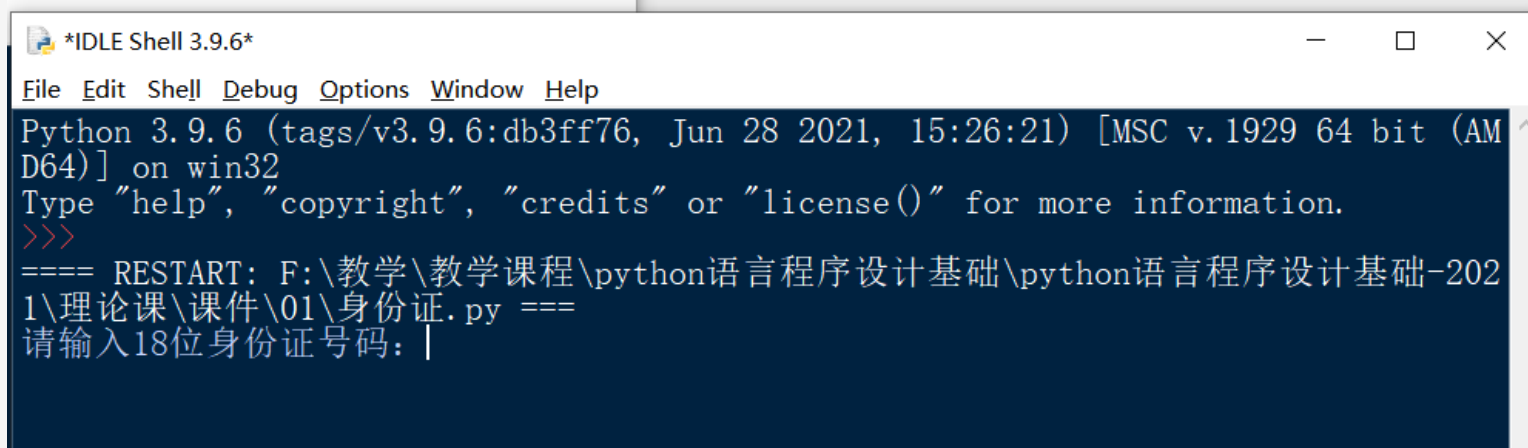
✓ 在系统命令行

✓ 使用IDLE



```
命令提示符 - python F:\教学\教学课程\python语言程序设计基础\python语言程序设计基础-2021\理论课\课件\01\身份证.py
Microsoft Windows [版本 10.0.19043.1165]
(c) Microsoft Corporation。保留所有权利。

C:\Users\mq>python F:\教学\教学课程\python语言程序设计基础\python语言程序设计基础-2021\理论课\课件\01\身份证.py
请输入18位身份证号码: _
```



```
*IDLE Shell 3.9.6*
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==== RESTART: F:\教学\教学课程\python语言程序设计基础\python语言程序设计基础-2021\理论课\课件\01\身份证.py ====
请输入18位身份证号码: |
```



内容回顾

- 机器语言、汇编语言、高级语言各自的特点是什么？
- 高级语言按照计算机执行方式的不同可分成哪两类？
- 简述编译和解释的区别
- Python语言的特点有哪些？
- Python语言的缺点有哪些？
- Python 的解释器种类以及相关特点？
- Python 2 和 Python 3 的区别有哪些？

■ 机器语言、汇编语言、高级语言各自的特点是什么？

- 机器语言

- 二进制语言，直接使用二进制代码表达指令

- 汇编语言

- 是二进制指令的文本形式，使用助记符与机器语言中的指令进行一一对应
 - 在不同的设备中，汇编语言对应着不同的机器语言指令集
 - 汇编语言：汇编指令按功能可分为四类：数据传送、算术运算、逻辑运算、控制转移

- 高级语言

- 接近自然语言的一种计算机程序设计语言

■ 高级语言按照计算机执行方式的不同可分成哪两类？

- 静态语言： C++/Java/C#
- 脚本语言： Python/PHP/ASP/Ruby/Perl/

■ 简述编译和解释的区别

• 编译

- 将源代码转换成目标代码的过程
- 源代码是高级语言代码，目标代码是机器语言代码
- 执行编译的计算机程序称为**编译器**
- 编译步骤包括：词法分析、语法分析、语义分析、汇编指令生成、转成机器码

• 解释

- 将源代码逐条转换成目标代码同时逐条运行目标代码的过程
- 执行解释的计算机程序称为**解释器**

■ Python语言的特点有哪些？

- **语法简洁**：实现相同功能，Python语言的代码行数仅相当于其他语言的1/10~1/15
- **与平台无关**：可以在任何安装解释器的计算机环境中执行，可不经修改地实现跨平台运行
- **粘性扩展**：可以集成C、C++、Java等语言编写的代码，通过接口和函数库等方式将它们整合在一起
- **开源且类库丰富**：<http://pypi.python.org/>，截止到2026年8月，已收录超过 88 万个第三方开源库
- **支持中文**：Python 3.0解释器采用 **UTF-8 编码**表达所有字符信息
- **强制可读**：Python通过强制**缩进**来体现语句间的逻辑关系，显著提高了程序的可读性
- **模式多样**：语法层面同时支持**面向对象**和**面向过程**两种编程方式

■ Python语言的缺点有哪些？

- 运行速度慢
- 代码不能加密

■ Python 的解释器种类以及相关特点？

- **Cpython**：官方版本的解释器，使用C语言开发，在命令行下运行python就是启动CPython解释器
- **IPython**：运行时底层依然依赖某个解释器（通常就是CPython）。它提供的是使用体验层面的增强

```
pip install ipython  
ipython
```



内容回顾

■ IPython

```
In [1]: 2 + 3
```

```
Out[1]: 5
```

不用写 print, 直接显示结果

```
In [2]: _ + 10
```

```
Out[2]: 15
```

① 下划线 = 上一次的结果 (也就是 5)

```
In [3]: Out[1] * 100
```

```
Out[3]: 500
```

② Out[1] = 第 1 行的结果, 往前翻多远都行



内容回顾

■ IPython

```
In [4]: name = "Tom"
```

```
In [5]: name.up<TAB>          # ③ 敲一半按 Tab 键，自动补全
        name.upper
```

```
In [6]: name.upper()
Out[6]: 'TOM'
```

```
In [7]: name.upper?          # ④ 加个问号 = 看说明书，不用去网上搜
        Docstring: Return a copy of the string converted to uppercase.
```

```
In [8]: %timeit 2 + 3        # ⑤ 百分号开头 = 魔法命令，这个是测速度的
8.2 ns per loop
```

```
In [9]: !ls                  # ⑥ 感叹号开头 = 直接敲系统命令
hello.py
```

■ Python 的解释器种类以及相关特点？

- **Cpython** : 官方版本的解释器，使用 C 语言开发，在命令行下运行 python 就是启动 Cpython 解释器
- **IPython** : 运行时底层依然依赖某个解释器（通常就是 CPython）。它提供的是使用体验层面的增强
- **Jython** : 运行在 Java 平台上的 Python 解释器，可以直接把 Python 代码编译成 Java 字节码执行
- **IronPython** : 和 Jython 类似，只不过 IronPython 是运行在 .Net 平台上的 Python 解释器，可以直接把 Python 代码编译成 .Net 的字节码



内容回顾

■ Python 2 和 Python 3 的区别有哪些？

- 打印时，py2需要可以不需要加括号，py3 需要

```
>>> print 'lili'
lili
```

```
>>> print('lili')
lili
```

- 输出中文的区别：Python2 输出中文 需加 `# -*- encoding:utf-8 -*-`

■ Python 2 和 Python 3 的区别有哪些？

- 读入内容：Python2 使用 `raw_input()`，Python 3 使用 `input()`。⚠ Python 中同样有 `input()` 函数，但表示的含义是：读入内容，当成代码执行

```
>>> x = input("请输入：")
```

```
请输入：1 + 1
```

```
>>> x
```

```
2
```

```
# 用户输的是文本，却被当成代码算出了结果
```




内容回顾

■ Python 2 和 Python 3 的区别有哪些？

- 整数类型：Python2 分 int 和 long 两种，超出范围会自动转成 long；Python3 只有 int 一种，大小没有上限

	int (定长整数)	long (长整数 / 大整数)
占用空间	固定，比如 64 位 = 8 字节	不固定，数越大占得越多
能表示的范围	有上限	没有上限

- 64 个二进制位只能组合出 2^{64} 种不同的数，去掉一半给负数，正数最大就是 9223372036854775807 ($2^{64} - 1$)
- 若再加 1，在 C 和 Java 里会溢出，变成最小的负数：

9223372036854775807 + 1 → -9223372036854775808

- ⚠ Python 2 不会溢出，它会自动把 int 转成 long



内容回顾

■ Python 2 和 Python 3 的区别有哪些？

- 整数类型：Python2 分 int 和 long 两种，超出范围会自动转成 long；Python3 只有 int 一种，大小没有上限

➤ 放回 Python 2 的语境：

```
# Python 2
```

```
>>> x = 9223372036854775807
```

```
>>> type(x)
```

```
<type 'int'>
```

还在范围内，是 int

```
>>> x + 1
```

```
9223372036854775808L
```

超出范围，Python 自动升级成 long

```
>>> type(x + 1)
```

```
<type 'long'>
```